

프리지아 랩

[Home](#)[Location](#)[Tags](#)[Media](#)[Guestbook](#)[Admin](#)[New Pc](#)

교육 & 세미나

Container & DevOps - DAY 3

by 강철 벚꽃 2026. 6. 24.

day3.zip
0.00MB

DAY 3

- DevOps 개념
- Source Control Management (SCM)
- 처음 만나는 GitHub
- GitHub Codespaces
- GitHub Actions

더 프리지아 랩

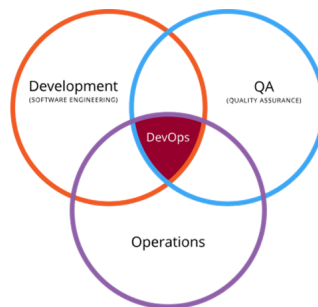
[Azure & Windows](#)[Windows 10](#)[Windows Server](#)[Azure](#)[PowerShell](#)[Localization Issue](#)[Programming](#)[Visual Studio](#)[Mixed Reality](#)[Windows Phone](#)[Exchange & Outlook](#)[Trouble Shooting](#)[System Administration](#)[Development](#)[Outlook Life](#)[My Favorite Tools](#)

DevOps 개념



DevOps의 개념

1. Development + Operation
2. 하나의 연속적인 일련의 프로세스로 통합하는 관례



66

DevOps 원칙

- ① 고객 중심으로 활동하기
- ② 끝을 염두에 두고 만들기
- ③ 처음부터 끝까지 책임지기
- ④ 완전히 독립적으로 다수의 역할로 일하는 자율 팀
- ⑤ 지속적으로 개선하기
- ⑥ 모든 것을 자동화하기

67

지속 통합 / 지속 제공(출시)

- | | |
|---|--|
| <ul style="list-style-type: none"> ▪ CI(Continuous Integration) <ol style="list-style-type: none"> ① 개발 단계에서 사용 ② 자동화된 코드 빌드 및 테스트 ③ 커밋 → 변경 검증 + 자동 패키징 | <ul style="list-style-type: none"> ▪ CD(Continuous Delivery) <ol style="list-style-type: none"> ① 제공 단계 자동화 ② 트리거를 통한 자동 배포 ③ 운영 환경으로 자동 게시 |
|---|--|

68

TechSmith

교육 & 세미나 

기술 세미나

메타넷

가톨릭 대학교

남부여성발전센터

미쓰비시전기오토메이션

미림 마이스터고

세완C교육

부산정보산업진흥원

Microsoft

NHN Cloud

Books

강철벼룩의 서재

출간 소식

번역 - 활자에 날을 세운다

LifeStyle INNOVATOR

경제지능 업그레이드

그들의 공간

가족 소식

경험 공유

Trend Or Future

공지사항

한국마이크로소프트

MAICPP Readines...

[LINC3.0사업단]알쓸챗
(Chat)잡 교육...클라우드 및 인공지능 기술
책 저자들과 함께...

한남 대학교 특강

제 11회 공가 SW 개발자 대
회 멘토로 위촉...

최근글 인기글

Container & DevOps -
DAY 3

2026.06.24



Container
& DevOp...
2026.06.24



Container
& DevOp...
2026.06.23



웹 프로그래밍 개요 - DAY 1
2026.06.22

"The
content f...
2026.01.01



최근댓글

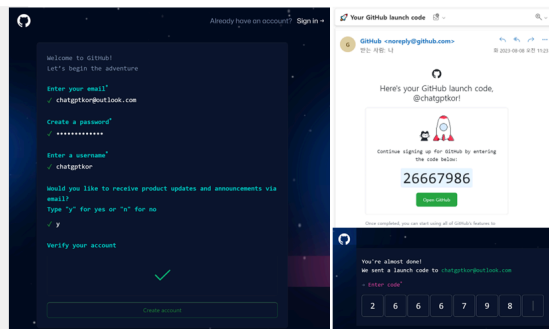
강철 베틀님 공감 꼭 누르...
물론이죠. 나가 연락할게요.
행복한 오늘 되세요~공감...
답변 정말 감사합니다 다음...

처음 만나는 GitHub



실습 | GitHub 가입 - 신원 확인

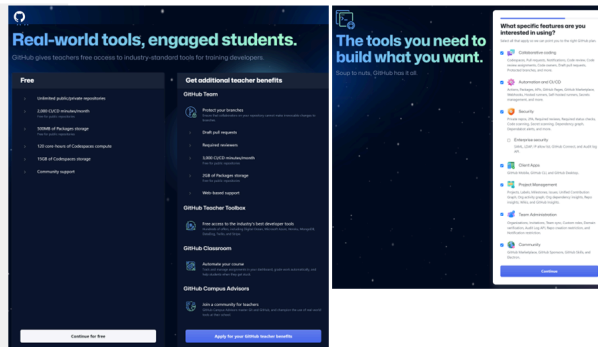
1. 기본 정보 입력
2. Launch Code 확인
3. Launch Code 입력



70

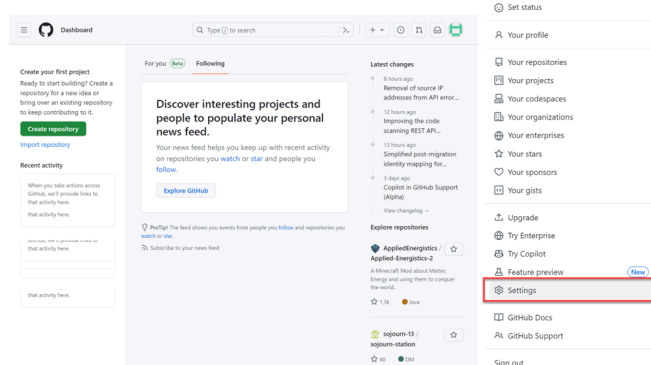
실습 | GitHub 가입 - 개인화

1. 팀 멤버와 학생 / 교수
2. 도구 선택
3. 무료 & 교수 혜택



71

처음 만나는 GitHub



72

태그

Developer Preview,
blob, 회의실, Azure AD,
정보문화사, uwp,
Windows 8,
windows 10, azure,
azure powershell,
Nano Server,
visual studio code,
윈도우 폰, 스타트업,
localization,
오프라인 주소록, RAS,
PowerShell, 실행모델,
header, 이웃룩 2007,
지앤선, VS2019,
코드검사분석, 윈도우 폰7,

실습 | 리포지토리 만들기

1. 리포지토리 메뉴

2. 리포지토리 만들기

- ✓ 이름
- ✓ 공개 유형
- ✓ README 파일 생성

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.

Required fields are marked with an asterisk (*)

1. Owner * Repository name *
chatgpt / demo-with-chatgpt
demo-with-chatgpt is available.

2. Public
Public Anyone on the Internet can see this repository. You choose who can commit.
Private You choose who can see and commit to this repository.

3. Initialize this repository with:
Add a README file
This repository will have a long description for your project. Learn more about READMEs.

Add gitignore
gitignore template None

Choose a license
Create a new one

4. Create repository

have any public repositories yet.

73

ca, Windcws Phone,
vscode, 지널링, lam

전체 방문자

1,096,717

Today : 94

Yesterday : 159

Source Control Management (SCM)



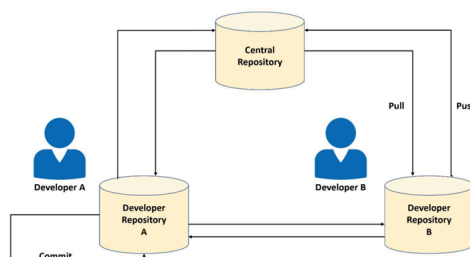
버전 제어

- ① 팀 개발 프로젝트를 위한 핵심 도구
- ② 코드 개발에서의 협업과 변경 추적
- ③ 강력한 소스 코드 버전 관리 도구, Git
 - ✓ 개발 머신에서 소스 리포지토리 사본 유지
 - ✓ 로컬에 모든 브랜치와 이력 정보 유지
 - ✓ 로컬과 소스 리포지토리 사이의 변경 공유
 - ✓ 네트워크 없이 로컬에서 변경 커밋과 버전 제어 가능
 - ✓ 개발 머신에서 브랜치 생성 및 병합, 게시, 폐기 가능

75

Git 워크플로

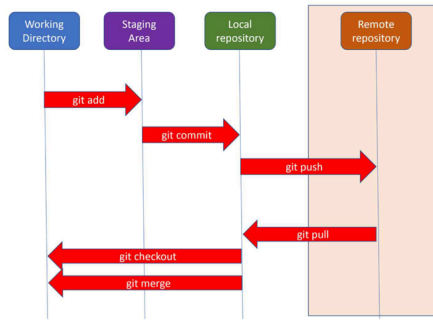
- ① 프로젝트의 리포지토리 만들기
- ② 로컬에 리포지토리 복제
- ③ 로컬에 새 파일 만들고 변경 저장
- ④ 원격 리포지토리에 변경 push
- ⑤ 원격의 변경 사항을 로컬로 pull
- ⑥ 변경을 로컬 리포지토리와 병합



76

Git 프로세스와 기본 명령

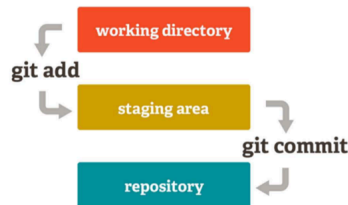
- ① 원격 리포지토리 로컬 복제
 - ✓ `git clone https://github.com/user/*.git`
- ② 프로젝트 수행
- ③ 작업을 로컬에 저장
 - ✓ `git add`
 - ✓ `git commit -m "commit message"`
- ④ 원격 리포지토리에 작업 저장
 - ✓ `git push`
- ⑤ 원격 리포지토리의 모든 업데이트 확인
 - ✓ `git pull`



77

Commit Flow

- ① 작업 디렉터리에서 소스 코드 파일 변경
- ② 스테이징 영역으로 변경 파일 추가
 - ✓ `git add`
- ③ 로컬 리포지토리에 새로운 커밋 만들기
 - ✓ `git commit`



78

Git 브랜치 작업 프로세스와 명령

- ① 새로운 브랜치 만들고 전환하기
 - ✓ `git checkout -b "branch1"`
- ② 새로운 기능 작업 수행
- ③ 로컬에 작업 저장
 - ✓ `git add`
 - ✓ `git commit -m "update from branch1"`
- ④ 원격 리포지토리 브랜치에 작업 저장
 - ✓ `git push`
- ⑤ 작업을 병합할 브랜치로 전환
 - ✓ `git checkout master`
- ⑥ 작업한 브랜치를 병합할 브랜치로 병합 후 원격 리포지토리에 저장
 - ✓ `git merge branch1`
 - ✓ `git push`

79

Branching Strategies

1. SCM 시스템에 저장된 코드의 버전
2. No Branch = 단일 코드 버전 = one flow

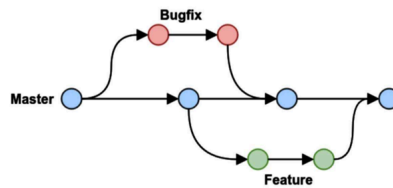


3. Git의 3가지 브랜치 전략
 - a. GitHub Flow
 - b. GitLab Flow
 - c. Git Flow

80

GitHub Flow

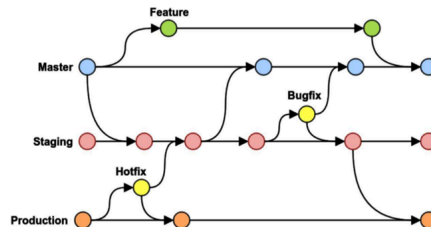
- 널리 사용되고 적용이 간단
- 새로운 기능 개발 시작 → 새로운 브랜치 + 정기적인 커밋
- 개발 작업 완료 → 브랜치 병합
- 운영에서 단일 버전의 코드 유지시 적용이 쉽다.



81

GitLab Flow

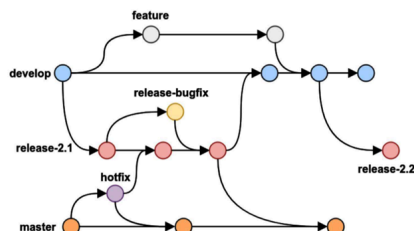
- 다중 환경 (Production, Staging, Development)
- 최소 3개의 브랜치 (master, staging, production)
- 안정적인 운영 릴리스 유지.



82

Git Flow

- 예정된 출시 사이클
- 릴리스 코드 → master
- 작업 중인 코드 → ex) develop
- 새로운 기능 추가 → ex) feature
 - ✓ Develop 브랜치에서 시작.
 - ✓ Feature를 develop으로 병합
- 기능 릴리스 → release
 - ✓ Develop 브랜치에서 시작
 - ✓ Release를 master로 통합 → develop로 병합
- 운영에서 버그 발견
 - ✓ master에서 fix 브랜치 만들어 작업
 - ✓ Fix를 master 또는 release 브랜치로 병합



83

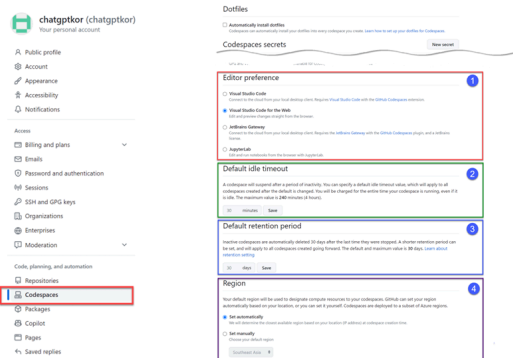
실습 - Git 기본 명령 다루보기

실습 - Git 브랜치 다루기

GitHub Codespaces



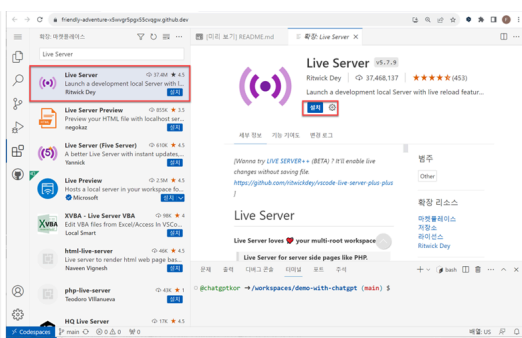
Codespaces 설정



87

실습 | Codespaces 만들기

1. Codespaces 만들기
2. Codespaces 구조
3. 확장 설치
 - ✓ Live Server



88

Codespaces의 OS 정보

command

bash

```
cat /etc/os-release
```

```

@chatgptkor → /workspaces/demo-with-chatgpt (main) $ cat /etc/os-release
NAME="Ubuntu"
VERSION="20.04.6 LTS (Focal Fossa)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 20.04.6 LTS"
VERSION_ID="20.04"
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
VERSION_CODENAME=focal
UBUNTU_CODENAME=focal
@chatgptkor → /workspaces/demo-with-chatgpt (main) $

```

89

Codespaces의 환경 정보

command

bash

```
printenv -0 | sort -z | tr '\0' '\n'
```

```

@chatgptkor → /workspaces/demo-with-chatgpt (main) $ printenv -0 | sort -z | tr '\0' '\n'
DESTDIR_FLAVOR=focal-scm
DOCKER_BUILDKIT=1
DOTNET_ROOT=/usr/local/dotnet/current
DOTNET_SKIP_FIRST_TIME_EXPERIENCE=1
DYNAMIC_INSTALL_ROOT_DIR=/opt
ENABLE_DYNAMIC_INSTALL=true
GOPATH=/go
GOROOT=/usr/local/go
GRADLE_HOME=/usr/local/sdkman/candidates/gradle/current
HOME=/home/codespace
HOSTNAME=codespace-b20af7
INTERNAL_VSCS_TARGET_URL=https://southeastasia.online.visualstudio.com
I18NRC=/usr/local/rvm/rubies/ruby-3.2.2/_librc
JAVA_HOME=/usr/local/sdkman/candidates/java/current
JAVA_ROOT=/home/codespace/java
JUPYTERLAB_PATH=/home/codespace/.local/bin
LANG=C.UTF-8
MAVEN_HOME=/usr/local/sdkman/candidates/maven/current
MAVEN_ROOT=/home/codespace/maven
MY_RUBY_HOME=/usr/local/rvm/rubies/ruby-3.2.2
NODE_ROOT=/home/codespace/nvm
NPM_GLOBAL=/home/codespace/.npm-global
NUGET_XMLDOC_MODE=skip
NVM_BSH=/usr/local/share/nvm/versions/node/v20.4.0/bin
NVM_CD_FLAGS=
NVM_DIR=/usr/local/share/nvm
NVM_INC=/usr/local/share/nvm/versions/node/v20.4.0/include/node

```

90

Codespaces의 Node 버전 변경 (일시적)

NVM

Node Version Manager

```
nvm install --lts
nvm use --lts
```

```

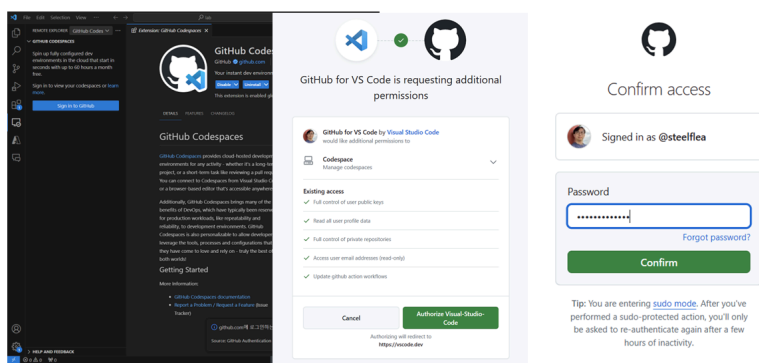
@chatgptkor → /workspaces/demo-with-chatgpt (main) $ node --version
v20.4.0
@chatgptkor → /workspaces/demo-with-chatgpt (main) $ nvm install --lts
Installing latest LTS version.
Downloading and installing node v18.17.1...
Downloading https://nodejs.org/dist/v18.17.1/node-v18.17.1-linux-x64.tar.xz...
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v18.17.1 (npm v9.6.7)
@chatgptkor → /workspaces/demo-with-chatgpt (main) $ nvm use --lts
Now using node v18.17.1 (npm v9.6.7)
@chatgptkor → /workspaces/demo-with-chatgpt (main) $ node --version
v18.17.1

```

91

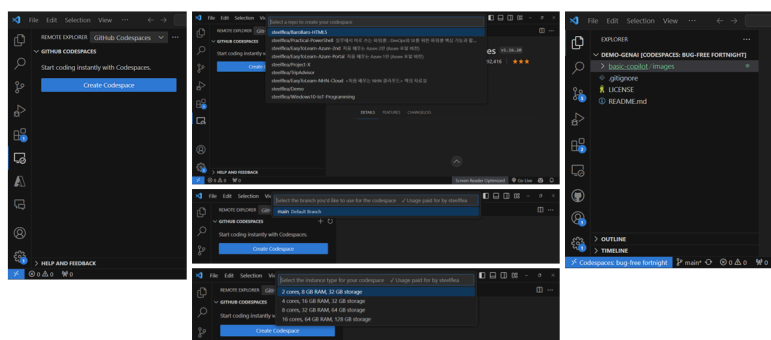
실습 – Codespaces에서 방명록 실행

Visual Studio Code + Codespaces 설정



93

VS Code에서 Codespaces 실행



94

GitHub Actions

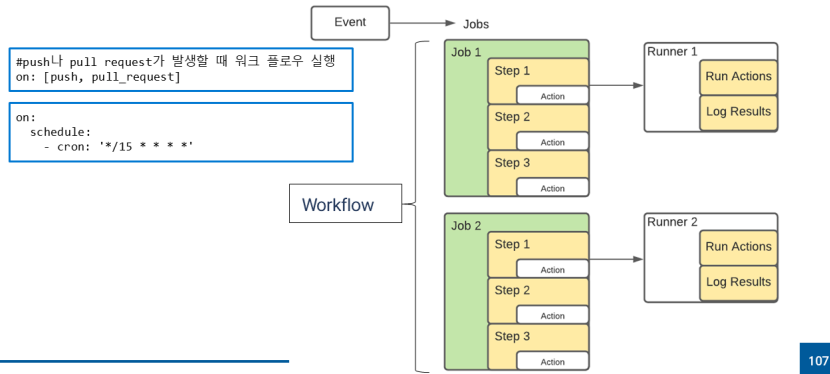


GitHub Actions 개요

- ① GitHub 저장소 기반 개발 Workflow 자동화 도구
- ② GitHub 내부에서 프로젝트 빌드, 테스트, 릴리즈, 배포 파이프라인 자동화
- ③ GitHub에서 제공하는 CI/CD 플랫폼
- ④ 리포지토리에 발생하는 이벤트 기반으로 Workflow 실행
- ⑤ Workflow 실행
 - ✓ GitHub이 제공하는 크로스 플랫폼 가상 머신에서 Workflow 실행
 - ✓ 자체 데이터 센터나 클라우드 인프라의 자체 호스팅 Runner

106

GitHub Actions 아키텍처



107

GitHub Actions의 구성

- ① 워크플로우(workflows)
 - ✓ 저장소에 추가하는 자동화된 프로세스
 - ✓ 하나 이상의 job, 이벤트에 의해 실행
- ② 이벤트 (Events)
 - ✓ 워크플로우를 실행하는 특정 활동이나 규칙
 - ✓ Commit의 Push, Pull Request
- ③ 러너 (runners)
 - ✓ GitHub 액션 러너 애플리케이션이 설치된 서버
 - ✓ GitHub에서 호스팅하는 러너 (Ubuntu Linux, Windows, macOS)
 - ✓ 직접 호스팅하는 러너
- ④ 작업 (jobs)
 - ✓ 워크플로우의 기본 단위
 - ✓ 더 작은 단위 스텝으로 구성
 - ✓ 여러 작업을 병렬 또는 순차적으로 실행 가능
- ⑤ 스텝 (steps)
 - ✓ 커맨드를 실행하는 독립적인 단위
 - ✓ 한 작업의 각 스텝은 동일 러너에서 실행
 - 작업의 액션들은 데이터 공유
- ⑥ 액션 (actions)
 - ✓ 워크플로우의 가장 작은 요소
 - ✓ 직접 만들거나 마켓에 등록된 액션 사용

108

YAML 형식

- ① 가독성 높은 구성 설정
- ② 키-값 쌍 정의
- ③ 중첩된 요소 / 블록
 - ✓ 기본 공백 2개
 - ✓ 탭 안됨
- ④ 키 결정
 - ✓ 들여쓰기를 점(.)으로 구분
- ⑤ 값 형식
 - ✓ 문자열
 - ✓ 정수
 - ✓ 불리언
 - ✓ 리스트

```
apiVersion: v1
kind: Service
metadata:
  labels:
    app: web
    name: web
spec:
  ports:
    - name: web-traffic
      port: 80
      protocol: TCP
      targetPort: 8080
  selector:
    app: web
  sessionAffinity: None
  type: LoadBalancer
```

109

실습 – GitHub Actions 구성 (코드)

민감한 정보 저장

비밀번호나 인증서 같은 민감한 정보
→ GitHub의 Secret으로 저장 후 환경 변수로 사용

```
jobs:
  example-job:
    runs-on: ubuntu-latest
    steps:
      - name: Retrieve secret
        # 환경변수로 저장하고
        env:
          super_secret: ${ secrets.SUPERSECRET }}
        # 저장한 환경변수를 활용한다.
        run: |
          example-command "$super_secret"
```

111

의존적인 작업 구성

다른 작업이 완전히 끝난 후 작업 실행
→ needs 키워드 사용
→ ex) build, test 작업이 이전 작업인 setup, build 작업이 끝난 후 실행되도록 설정

```
jobs:
  setup:
    runs-on: ubuntu-latest
    steps:
      - run: ./setup_server.sh
  build:
    needs: setup
    runs-on: ubuntu-latest
    steps:
      - run: ./build_server.sh
  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: ./test_server.sh
```

112

빌드 매트릭스 활용

워크 플로우가 다양한 OS, 플랫폼, 언어의 여러 조합에서 테스트 실행하는 경우.
→ strategy 키워드로 빌드 옵션을 배열로 받는다.

```
jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        # 다양한 버전의 Node.js를 이용하여 작업을 여러번 실행
        node: [6, 8, 10]
    steps:
      - uses: actions/setup-node@v1
        with:
          node-version: ${ matrix.node }}
```

113

종속성 캐싱

GitHub의 러너는 각 작업에서 새로운 환경으로 실행
→ 작업들이 종속성을 재사용하는 경우, 파일들을 캐싱해 성능을 높인다.
→ 캐시를 생성하면 해당 리포지토리의 모든 워크 플로우에서 사용 가능

```
jobs:
  example-job:
    steps:
      - name: Cache node modules
        uses: actions/cache@v2
        env:
          cache-name: cache-node-modules
        with:
          # ~/.npm 디렉토리를 캐시해 성능을 높인다
          path: ~/.npm
          key: ${ runner.os }}-build-${ env.cache-name }}-${ hashFiles('**/package-lock.json') }}
          restore-keys: |
            ${ runner.os }}-build-${ env.cache-name }}-
```

114

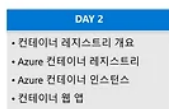
실습 - GitHub Actions 구성 (컨테이너)

[공감](#)[구독하기](#)

'교육 & 세미나' 카테고리의 다른 글

Container & DevOps - DAY 2 (0)	2026.06.24
Container & DevOps - DAY 1 (0)	2026.06.23
웹 프로그래밍 개요 - DAY 1 (0)	2026.06.22

관련글



[Container & De...](#) [Container & De...](#) [웹 프로그래밍 개...](#)

프리지아 랩

프리지아(Phrygia)는 소아시아의 고대 지역으로 수도는 고르디온이며, '손에 닿는 것을 모두 황금으로 바꾸'는 미다스왕의 전설로 유명하다. 프리지아 랩은 스마트 라이프를 추구하는 라이프스타일 INNOVATOR다. 사소하고 일상적인 것을 새로운 가치로 재탄생 시킨다.

[구독하기](#)

댓글 0



이름

.....

내용을 입력하세요.

댓글

Designed by 티스토리

© AXZ Corp.

